

Introduction and Motivation

Constructing an optimal batting order is a fundamental and recurring challenge in baseball, faced by managers at every level of the sport. Traditionally, lineup construction has been guided by a mixture of statistical reasoning and longstanding heuristics (for example, placing high on-base percentage hitters at the top of the order and power hitters in the middle). However, a lineup determines not only the sequence in which players bat, but also how often high-impact hitters come to the plate and how effectively offensive opportunities are converted into runs. These conventions, while intuitive, often fail to fully leverage the depth and granularity of modern player performance data. With the advent of Major League Baseball's Statcast system, teams now have access to highly detailed, pitch-by-pitch data capturing nuanced aspects of player performance, such as exit velocity, launch angle, and expected outcomes of batted balls. This creates an opportunity to explore whether machine learning methods can provide a more systematic, data-driven approach to lineup construction, potentially uncovering patterns not immediately apparent through conventional analysis.

In this project, we develop a two-model machine learning framework to study and predict batting order decisions. The first model operates at the player level, using offensive performance metrics to predict a probability distribution over the nine possible lineup positions. The second model builds on this representation by considering groups of nine players simultaneously, learning to predict a complete lineup configuration. By leveraging learned player embeddings and modeling interactions across players within a lineup, this approach aims to better approximate the joint decision-making process underlying real-world lineup construction. This framework extends prior work in operations research that models baseball as a probabilistic

system, where lineup construction can be analyzed as an optimization problem, and player sequencing influences run production.¹ While these approaches have provided valuable insights, they typically rely on fixed probabilistic assumptions.² In contrast, our method uses data-driven representations learned directly from high-dimensional Statcast features, allowing for greater flexibility in capturing complex, nonlinear relationships between player performance and lineup placement.

Our analysis is driven by two primary research questions. First, how accurately can we predict a player's lineup position based solely on their performance metrics up to a given point in the season? Second, which player attributes are most influential in determining lineup placement? More broadly, we seek to understand the extent to which lineup decisions in professional baseball can be explained and potentially replicated through statistical learning methods. The motivation for this work stems from both a personal passion for baseball and the richness of its data. From a machine learning perspective, the problem presents an interesting combination of classification and structured prediction tasks, while the sport itself still relies heavily on human judgment in strategic decisions. By applying modern learning techniques to lineup construction, we aim to bridge the gap between data availability and decision-making, providing insight into how quantitative methods can complement or challenge traditional approaches to the game.

¹ Bukiet, Bruce, et al. "A Markov Chain Approach to Baseball." *Operations Research*, vol. 45, no. 1, Feb. 1997, pp. 14–23. pubsonline.informs.org (Atypon), <https://doi.org/10.1287/opre.45.1.14>.

² "Stochastic Modeling and Optimization in Baseball." *Wiley Encyclopedia of Operations Research and Management Science*, by James J. Cochran et al., 2011. DOI.org (Crossref), <https://doi.org/10.1002/9780470400531.eorms0836>.

Data Description

The data for this project was sourced using the Python package `pybaseball`, which provides programmatic access to several major baseball databases, including Baseball Savant, Baseball Reference, and FanGraphs.³ Specifically, we used the `statcast` function to retrieve pitch-by-pitch data for the entire 2025 Major League Baseball season. The raw Statcast dataset contains 118 variables per pitch, including pitch characteristics (location, velocity, spin rate), game context, player identifiers, and batted ball outcomes. To ensure consistency and representativeness of player performance, we restricted the dataset to players with at least 100 plate appearances during the season. We also standardized player identifiers using the `playerid_reverse_lookup` function, allowing us to consistently match players across datasets.⁴ In addition to Statcast data, we incorporated a game log dataset containing the batting order for each team in each game, including game date and home/away team designations. These two sources were merged using shared identifiers such as game date and team.

The unit of analysis in our final dataset is a player-game instance, where each row represents a single player's performance entering a specific game, along with their observed lineup position in that game. To construct meaningful input variables, we transformed pitch-level data into cumulative player statistics calculated up to each game date. These features were designed to capture multiple dimensions of offensive performance, including traditional rate statistics such as batting average (AVG), slugging percentage (SLG), and on-base percentage (OBP), and more advanced metrics like expected weighted on-base average (xwOBA). These

³ <https://github.com/jldbc/pybaseball/tree/master/docs>

⁴ Tools such as the Chadwick Register (which can also be accessed through `pybaseball`) offer easy ways to choose between ID methods. The most up-to-date version is MLBID, but we choose to use Retrosheet which is more commonly used for historical data since that was the key type used by our game log of every lineup.

variables were selected to provide a balanced representation of hitting ability, plate discipline, and quality of contact.

To prevent data leakage and better reflect real-world decision-making, all features were computed as cumulative statistics up to, but not including, the current game. This was implemented using cumulative sums with a one-step shift. Additional data cleaning steps included removing rows with missing lineup positions (e.g., pinch hitters), ensuring there was always only one observation per player per game, and filtering out each player's first 10 games of the season to allow performance metrics to stabilize. The final dataset consists of 36,665 player-game observations across the 2,430 games of the 2025 MLB season. Each observation includes 11 engineered performance features, contextual variables such as game date, team, and player ID, and the target variable representing lineup position (1–9).

Methods

We approach lineup prediction as a two-part machine learning problem, combining both multi-class classification at the player level and structured prediction at the team level. The input features consist of eleven cumulative offensive metrics, and the target variable is lineup position, encoded from 0 to 8 (Fig. 1). To preserve the real-world structure of lineups and prevent data leakage, we perform a group-based train/validation split that ensures all nine players from a given game remain in the same partition. This is critical because lineup decisions are inherently joint, and splitting players across sets would leak information. Input features are standardized to improve numerical stability and convergence during training. Both models are trained using mini-batch gradient descent, with batch sizes of 32 for the player model and 16 for the team model. This is due to the smaller number of samples for the team model to train on (428 full

lineups vs 36,665 individual player features) since we only included a game instance in the team model's input set if it had valid entries for all 9 players. These preprocessing steps ensure that both models operate on consistent, well-structured inputs that reflect real game conditions.

The player-level model predicts the likelihood of each lineup position for an individual player using a feedforward neural network. The architecture consists of fully connected layers of sizes 128, 64, and 32, followed by a final output layer of size 9, with ReLU activations applied between layers. The model outputs logits, which are converted to probabilities via the softmax function in the cross-entropy loss. This setup is appropriate because lineup positions are mutually exclusive, and the softmax produces a probability distribution over all nine positions. A key design choice is the inclusion of a 32-dimensional embedding layer, which captures a compact representation of player skill and is later reused in the team-level model. The model is trained using the Adam optimizer with weight decay over 100 epochs, and the best version is selected based on minimum validation loss. Performance is evaluated using exact accuracy, top-3 accuracy, and mean absolute error (MAE), the latter capturing the ordinal distance between predicted and true positions. These metrics reflect the fact that many lineup spots have similar player profiles, especially in the middle of the lineup, making near-misses still informative.

The team-level model builds on this representation by predicting complete batting orders for groups of nine players. Each lineup is passed through the pretrained player model to obtain embeddings, which are then fed into a linear scoring layer that produces position scores for each player. During training, cross-entropy loss is computed independently for each of the nine players and averaged across the lineup. To construct a valid lineup at inference time, we apply the Hungarian algorithm (linear sum assignment) to the predicted probabilities, ensuring that each position is filled exactly once while maximizing the overall assignment score. This allows

the model to produce a globally optimal lineup rather than independent predictions. Model performance is evaluated using exact match percentage, average positional displacement, and pairwise accuracy, which measures whether the model correctly predicts the relative ordering of players. We also compute overall player placement accuracy, position-wise accuracy, and a confusion matrix to better understand prediction patterns. Finally, feature importance is assessed using permutation importance based on changes in MAE, identifying which performance metrics most strongly influence predictions. Together, these methods allow us to evaluate both individual prediction accuracy and the model's ability to reconstruct full lineup structures, which we analyze in the following section.

Results

As seen in the model evaluation output, the player-level model demonstrates that lineup position can be predicted with meaningful accuracy using offensive performance metrics (Fig. 2). It achieves an exact accuracy of 51.28% and a mean absolute error of 1.01. We also evaluated the model's ability to have the true lineup position within its top 3 predictions, resulting in a proportion of 83.78% of players. Additionally, training loss decreased steadily to just above 1.0 over 100 epochs, while the validation loss achieved a minimum of approximately 1.3 and showed limited improvement beyond that point (Fig. 3). In the context of a nine-class classification problem using cross-entropy loss, this gap suggests that the model successfully captures structure in the training data but begins to overfit, with diminishing returns in generalization performance after early epochs. These results suggest that while exact classification across nine possible positions is inherently difficult, the model captures strong signals about where a player generally belongs in the batting order. The second, team-based model reflects the increased difficulty of predicting all nine positions simultaneously. It achieves

an exact match rate of 12.38%, which is low but expected given the combinatorial complexity of full lineup prediction ($9!=362,880$). More informative metrics help us show this relatively strong performance, with an average displacement of 0.76 positions per player and a pairwise accuracy of 87.66% (Percentage of pairs of 2 players in which the model correctly guessed which player batted before the other). In other words, even when exact positions differ, the model typically places players in approximately the correct part of the lineup. Position-specific accuracy further supports this, with top-of-the-order spots predicted most accurately and middle-to-lower positions showing more variability.

Interpreting these results reveals that both models have learned patterns consistent with established baseball strategy. Feature importance analysis from the player model shows that HardHit%, xwOBA, and HR are the most influential predictors, suggesting that quality of contact and power are key drivers of lineup placement. This aligns with the tendency for high-impact hitters to appear in the middle of the order. At the same time, relatively strong accuracy at the top of the lineup indicates that the team model also captures the importance of on-base and contact-oriented metrics for leadoff and early-order roles, whereas lower lineup positional roles are less clearly defined and more dependent on contextual factors not included in the data. Despite this, several of our earlier approaches did not perform as well by these metrics and subsequently informed the final model design. Notably, we had to tune the thresholds by which we filtered out players that did not appear in enough games. However, this had to be balanced with the need for as much data as possible in order to maximize sample count for the team-based lineup prediction model (which would only use a sample if all 9 position slots were filled with a valid player feature entry). On the modeling side, earlier versions that treated lineup positions as independent classes without accounting for their ordinal relationships produced less

stable training behavior and larger positional errors, even with a much larger epoch count (Fig. 4). Through repeated testing and tuning of model parameters, these issues were addressed and improved overall predictive performance.

Discussion and Conclusion

The results of this study suggest that lineup construction in baseball (at the professional level and otherwise) is both more predictable and complex than traditional intuition might imply. At the player level, the model demonstrates that offensive performance metrics contain strong predictive signals about where a player is likely to bat. Achieving over 50% exact accuracy and 83% top-3 accuracy indicates that, while precise placement is challenging, the model can reliably identify the general region in the lineup that a player belongs to. These findings support the known idea that lineup decisions are not arbitrary, and reflect the minor changes that might be made to the batting order from one game to another. At the same time, there are some limitations we have observed in this model's performance. Notably, the gap between training and validation loss in Figure 3 suggests some overfitting based on noise in the dataset, and may lead to the model's limited ability to generalize its predictions to unseen games (i.e., from other seasons).

At the team level, prediction difficulty increases substantially due to the combinatorial nature of accurately constructing a full lineup. Although the exact match rate is low, more informative metrics show that the model performs relatively well, with high pairwise accuracy and low average positional displacement (Fig. 2). The overall player placement accuracy of nearly 60% and position-wise results further support this, and reveal a clear trend: Lineup positions towards the top of the order are predicted most accurately and consistently, while middle and lower positions exhibit more variability. This can be improved upon in future iterations of this approach by also including the pitcher and batter handedness in the input

features. These values can serve as a key piece of context for determining a batting order each day, as right-handed batters typically perform better against left-handed pitchers and vice versa. By avoiding a complete reliance on static season statistics, these models can learn to mirror real-world strategy, where later positions can be more flexible and influenced by contextual or managerial factors not captured in the data.

The importance of the input features confirms that the model is making meaningful decisions based on established baseball knowledge rather than overfitting based on random statistics. Our analysis shows that hard hit percentage and xwOBA serve as the main predictive metrics for lineup construction, which makes sense: These statistics have nothing to do with the result of the swing but show the rate at which a player is expected to produce positive outcomes. The better players are at these metrics, the higher up in the lineup they should be to maximize their at-bats. On-base percentage is found in the middle of the pack in terms of feature importance (since it is viewed as critically important for lineup positions 1 and 2 but much less for others), while strikeout-to-walk ratio is found at the very bottom of our feature importances. This is a statistic that is very polarizing when it comes to hitter profiles, and while some hitters excel because of their raw hitting ability, many are productive because of their excellent batting eye. This is a feature that we may replace in future iterations with a hitter's zone contact percentage, or the measure of how often they connect with a ball that passes through the strike zone. This is much more predictive of a hitter's ability than a statistic that relies on pitchers throwing strikes. Even so, the player model's predictive abilities show that these features allow for batter profiles to be created for each region of the lineup.

Regardless of the relative success of this two-model approach, we are also constrained by the fact that the game is always changing. New rules, such as the automated ball and strike

challenge system (ABS), were introduced in the 2026 season and further limit the model's predictive capabilities. Feature importance will likely be affected as a result of this, and new player metrics such as challenge success rate may be taken into account when building a batting order. Furthermore, only training on players (and lineups made up of those players) who have played over ten games in the MLB is a key limitation. At first glance, this seems fine as the MLB's season is 162 games long, but when factoring in injuries, prospect call-ups, and other managerial decisions, the number of player-game instances available to train on is significantly limited. This disproportionately affects worse teams, such as the Chicago White Sox, who had 11 players debut for them in 2025, while established dynasties like the Los Angeles Dodgers were able to keep the same rotation of top-tier players all season. This may have allowed the models to train more on some players than others, even when only looking at their season statistics. Overall, this study demonstrates that machine learning can meaningfully approximate lineup construction decisions, offering a data-driven complement to traditional baseball strategy while highlighting areas for further exploration. While season-long averages provide a strong foundation for identifying a specific player's lineup utility, future iterations should account for situational volatility to improve on the instant real-world applications.

Figures

	AVG	SLG	OBP	HR	xwOBA	K%	BB%	K/BB	Contact %	Sweet Spot%	HardHit %	lineup _pos
count	36665	36665	36665	36665	36665	36665	36665	36665	36665	36665	36665	36665
mean	0.2483	0.4087	0.3192	8.5476	0.3156	0.2168	0.0838	3.1411	0.7832	0.3600	0.4200	4.7999
min	0.0536	0.0714	0.0980	0.0000	0.0984	0.0000	0.0000	0.0000	0.3902	0.0800	0.0435	1.0000
25%	0.2262	0.3559	0.2928	3.0000	0.2871	0.1707	0.0584	1.8500	0.7398	0.3280	0.3650	3.0000
50%	0.2500	0.4053	0.3190	6.0000	0.3165	0.2192	0.0805	2.6000	0.7808	0.3610	0.4304	5.0000
75%	0.2725	0.4608	0.3462	12.000	0.3442	0.2602	0.1058	3.7143	0.8293	0.3924	0.4773	7.0000
max	0.4667	1.0968	0.5909	60.000	0.6318	0.6098	0.3256	38.000	1.0000	0.7368	0.7931	9.0000
std	0.0398	0.0866	0.0437	7.8731	0.0472	0.0653	0.0344	2.2366	0.0653	0.0543	0.0876	2.5424

Figure 1: Summary Statistics for the 11 target variables and 1 outcome variable in our final feature set.

PLAYER MODEL: Exact Accuracy: 0.5128 Top-3 Accuracy (Actual position in top 3 predicted positions): 0.8378 MAE: 1.0057 Feature Importances: HardHit%: 0.5938 xwOBA: 0.5478 HR: 0.5019 SweetSpot%: 0.4959 Contact%: 0.4287 SLG: 0.4128 OBP: 0.4020 BB%: 0.3880 K%: 0.3787 AVG: 0.3117 K/BB: 0.2027	TEAM MODEL: Exact Match %: 0.1238 Avg Displacement/Position Error: 0.7617 Pairwise Accuracy (Predicting which player in each pair of 2 bats before the other): 0.8766 Overall Player Placement Accuracy: 2294/3852 (59.55%) Lineup Position Accuracy: Position 1: 82.71% Position 2: 75.93% Position 3: 72.90% Position 4: 60.51% Position 5: 48.13% Position 6: 42.06% Position 7: 42.52% Position 8: 46.73% Position 9: 64.49% Confusion Matrix: True Position (Rows) vs Predicted Position (Columns): <table><tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr><tr><td>1</td><td>354</td><td>25</td><td>9</td><td>3</td><td>14</td><td>13</td><td>1</td><td>4</td><td>5</td></tr><tr><td>2</td><td>18</td><td>325</td><td>23</td><td>16</td><td>20</td><td>7</td><td>7</td><td>6</td><td>6</td></tr><tr><td>3</td><td>12</td><td>21</td><td>312</td><td>46</td><td>15</td><td>13</td><td>5</td><td>2</td><td>2</td></tr><tr><td>4</td><td>6</td><td>12</td><td>39</td><td>259</td><td>69</td><td>29</td><td>7</td><td>5</td><td>2</td></tr><tr><td>5</td><td>7</td><td>12</td><td>17</td><td>65</td><td>206</td><td>69</td><td>26</td><td>19</td><td>7</td></tr><tr><td>6</td><td>10</td><td>10</td><td>13</td><td>24</td><td>66</td><td>180</td><td>76</td><td>30</td><td>19</td></tr><tr><td>7</td><td>13</td><td>11</td><td>6</td><td>7</td><td>22</td><td>68</td><td>182</td><td>92</td><td>27</td></tr><tr><td>8</td><td>5</td><td>4</td><td>6</td><td>6</td><td>8</td><td>30</td><td>85</td><td>200</td><td>84</td></tr><tr><td>9</td><td>3</td><td>8</td><td>3</td><td>2</td><td>8</td><td>19</td><td>39</td><td>70</td><td>276</td></tr></table>		1	2	3	4	5	6	7	8	9	1	354	25	9	3	14	13	1	4	5	2	18	325	23	16	20	7	7	6	6	3	12	21	312	46	15	13	5	2	2	4	6	12	39	259	69	29	7	5	2	5	7	12	17	65	206	69	26	19	7	6	10	10	13	24	66	180	76	30	19	7	13	11	6	7	22	68	182	92	27	8	5	4	6	6	8	30	85	200	84	9	3	8	3	2	8	19	39	70	276
	1	2	3	4	5	6	7	8	9																																																																																												
1	354	25	9	3	14	13	1	4	5																																																																																												
2	18	325	23	16	20	7	7	6	6																																																																																												
3	12	21	312	46	15	13	5	2	2																																																																																												
4	6	12	39	259	69	29	7	5	2																																																																																												
5	7	12	17	65	206	69	26	19	7																																																																																												
6	10	10	13	24	66	180	76	30	19																																																																																												
7	13	11	6	7	22	68	182	92	27																																																																																												
8	5	4	6	6	8	30	85	200	84																																																																																												
9	3	8	3	2	8	19	39	70	276																																																																																												

Figure 2: Text-based evaluation output from both player and team models.



Figure 3: Visualization showing progression of loss values for training and validation sets through 100 epochs.

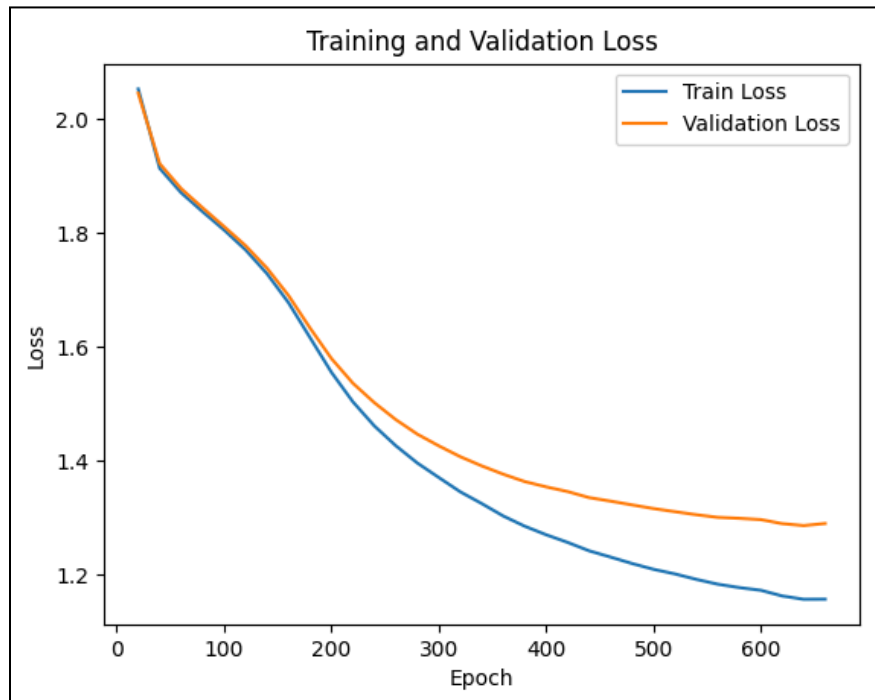


Figure 4: Older version of loss visualization before implementing an ordinal relationship between lineup positions (Mispredicting a leadoff hitter as a 2-hitter would be penalized the same as predicting them as the last batter).